

CHAPTER 1

PREMBLE

1.1 INTRODUCTION

The Smart Blood Donating App and Emergency Request System is a real-world healthcare application developed using Python that helps patients quickly find blood donors during emergencies. The main aim of this project is to reduce the delay in finding blood and improve communication between hospitals, patients, and blood donors. In this system, blood donors first register by entering their details such as name, blood group, phone number, and location. All donor information is securely stored in a database. Whenever there is an emergency blood requirement, the admin or hospital can create a blood request by selecting the required blood group and location.

After the request is created, the system automatically sends SMS notifications to matching donors. Interested donors can accept the request through the application or confirmation message. Once a donor accepts the request, the emergency request is automatically closed to avoid duplicate responses and confusion. This makes the system fast, smart, and efficient during critical situations. In many emergency situations, hospitals and patients face difficulty in finding the required blood group at the right time. Delays in communication and manual searching can sometimes become life-threatening.

This project is developed to solve that problem by creating a smart platform where blood requests can be managed efficiently and donors can be contacted instantly. The system checks the database and identifies donors with matching blood groups available near the requested location. The project mainly focuses on automation, speed, and reliability in emergency healthcare support. It helps hospitals, blood banks, and patients manage blood requests digitally instead of depending on phone calls or manual records.

The system can also maintain donor history, request status, and blood availability details, making the process more organized and transparent. The Smart Blood Donating App is a socially beneficial project because it supports emergency healthcare services and encourages voluntary blood donation. It is an excellent project for software engineering students as it combines real-world problem solving, database management, backend development, and communication technologies into a single practical application.

SMART BLOOD DONATION AND EMERGENCY REQUEST



Fig 1.1: Blood Donation

1.2 OVERVIEW

The Smart Blood Donating and Emergency Request App is a centralized system that manages blood donor information and emergency blood requests through an automated platform. The project is designed to simplify the process of finding blood donors and improve emergency response using technology. A major functionality of the project is the automated communication process. Instead of manually contacting donors one by one, the system instantly sends notifications or SMS messages to all matching donors. Donors who are willing to donate can accept the request through the system. After successful acceptance, the request status changes to closed, ensuring proper management of emergency cases. The Smart Blood Donating and Emergency Request App is a modern healthcare support system developed using Python to improve the process of blood donation and emergency blood management. The project is designed to create a fast, reliable, and user-friendly platform

One of the most important features of the system is its automated notification mechanism. Instead of searching for donors manually, the system instantly sends SMS alerts or notifications to suitable donors. This project is highly useful in real-life healthcare environments because it supports faster emergency response and encourages voluntary blood donation

1.3 PROBLEM DEFINITION

The lack of a centralized and automated blood donation system makes it difficult to quickly find suitable donors during emergencies. Manual communication methods are time-consuming and inefficient, leading to delays in emergency response. Therefore, a smart system is required to manage donor records, send instant notifications, and handle blood requests efficiently.

1.4 MOTIVATION

The motivation for developing the Smart Blood Donating and Emergency Request System is to provide a reliable and technology-driven solution for managing blood donation services efficiently during emergency situations. In critical medical cases, delays in finding suitable blood donors can create serious risks for patients and increase the burden on healthcare organizations. Existing manual methods of communication are often slow, unorganized, and ineffective in handling urgent blood requests.

Another major motivation is to encourage voluntary blood donation and create a centralized platform that connects donors, hospitals, and patients in a faster and more organized manner. The project demonstrates how software solutions can contribute to improving healthcare services and saving human lives.

1.5 OBJECTIVE

The objective of the Smart Blood Donating and Emergency Request System is to develop an efficient and automated platform using Python for managing blood donation activities during emergencies. The system aims to simplify donor registration, quickly identify suitable donors, send instant emergency notifications, and manage blood requests effectively. It also focuses on reducing response time, improving communication between donors and patients, and providing a reliable solution for emergency blood management. To develop a Smart Blood Donating and Emergency Request System using Python.

To create a donor registration module for storing donor details and blood group information.

To provide quick identification of matching blood donors during emergencies.

To automate the process of sending emergency blood request notifications.

To manage donor responses and automatically close requests after acceptance.

To maintain donor and request records efficiently using Python-based database handling.

To reduce the time required for finding blood donors and improve emergency response.

To build a user-friendly and reliable application for blood donation management.

1.6 SCOPE OF PROJECT

The scope of the Smart Blood Donating and Emergency Request System is to provide an automated and efficient platform for managing blood donation activities using Python. The system is designed to identify suitable donors based on blood group and availability, allowing faster response during critical situations.

Donor Registration – Enables donors to enter and manage their personal details, blood group, and contact information in the system.

Database Management – Stores and maintains donor and emergency request records securely using Python.

Emergency Request Creation – Allows hospitals or patients to generate urgent blood requests when blood is required.

Blood Group Matching – Identifies suitable donors based on the required blood type and donor availability.

SMS Notification Service – Sends instant emergency alerts to matching donors through automated messaging.

Donor Response Handling – Allows donors to accept or reject emergency blood requests easily.

Efficient Communication – Improves coordination between donors, patients, and hospitals during emergencies.

Time Reduction – Minimizes delays in finding blood donors and increases emergency response speed.

CHAPTER 2

LITERATURE SURVEY

2.1 RELATED WORK

[1] .Optimization of blood donation activity.

Published in: Applied Medical Informatics , 2018, Directory of open access journals.

By: Anca-Elena ; Adrina Albu.

[2] .Smart approaches for encouraging the blood donation.

Published in: Asian journal of transfusion science , 2024 , directory of open access journals.

By: Rakesh Sharma; Debadri Banerjee; Anupama Singh; Vikas Anand Saharan.

[3] .Design of a smart blood donation assistant based on large language model.

Published in: Zhongguo shuxue zazhi, 2026, Directory of open access journals.

By: LUN Lan; WAN Kangile; ZHENG Yue; ZHAO Xiaoya; HAN Zhedong.

2.2 EXISTING SYSTEM

The existing blood donation system mainly depends on manual methods such as phone calls, personal contacts, hospital records, and social media platforms to find blood donors during emergencies. Donor information is often maintained in separate records without proper centralization, making it difficult to quickly identify suitable donors based on blood group and availability.

Communication in the existing system is time-consuming and lacks automation, which can delay emergency response and create difficulties for patients and hospitals. There is also no proper mechanism to send instant notifications, track donor responses, or automatically close requests after donor confirmation. As a result, the existing system is less efficient, less organized, and unreliable during critical medical situations.

2.2.1 ADVANTAGES :

1. Life-Saving Emergency Support System
2. Faster Access to Blood During Critical Situations
3. Instant Communication with Blood Donors
4. Efficient and Automated Blood Request Management
5. Quick Identification of Suitable Donors
6. Improved Coordination Between Hospitals and Donors
7. Reduced Delay in Emergency Medical Treatment
8. Secure and Centralized Donor Information Management
9. Minimized Manual Effort and Human Errors
10. Reliable and Technology-Driven Healthcare Solution

2.2.2 .DISADVANTAGES :

1. Wrong donor information may cause issue.
2. Depends on internet and network services.

2.3 PROPOSED SYSTEM:

The proposed Smart Blood Donating and Emergency Request System is an advanced application developed in Python to provide a faster and more efficient method for handling blood donation services. The system replaces traditional manual processes with an automated platform that manages donor information, emergency blood requests, and communication between donors and healthcare organizations.

The application allows blood donors to create and maintain their profiles by registering details such as blood group, contact number, and location. These details are stored in a centralized database that can be accessed whenever an emergency blood requirement occurs. During emergencies, the system automatically identifies eligible donors based on the requested blood group and sends instant SMS notifications to them.

Donors can respond to the emergency request through the application, making the process simple and efficient. After a donor accepts the request, the system automatically updates the request status and closes it to avoid multiple responses for the same requirement. The proposed system aims to provide a reliable, secure, and user-friendly healthcare support solution while minimizing manual effort and improving emergency medical services through Python-based automation.

Advantages of the Proposed System

- 1.Fast Emergency Response.
- 2.Smart Donor Matching.
- 3.Reduced Manual Work
- 4.Secure Data Storage
- 5.Reliable Blood Donation System.

CHAPTER 3

SYSTEM REQUIREMENT SPECIFICATION

3.1 SOFTWARE REQUIREMENTS

1. Programming Language – Python 3.x
2. IDE/Code Editor – Visual Studio Code / PyCharm
3. Database Type – In-Memory Python Lists (No external database required)
4. Operating System – Windows / Linux / macOS
5. Execution Environment – Python Console / Terminal
6. Input Method – Keyboard Input
7. Output Method – Console Display
8. SMS Functionality – Simulated SMS Notification System

3.2 HARDWARE REQUIREMENTS

1. Processor – Dual Core Processor or Higher.
2. RAM – Minimum 4 GB RAM.
3. Storage – Minimum 500 MB Free Disk Space.
4. Monitor – Standard Display Monitor.
5. Input Devices – Keyboard and Mouse.
6. Network – Internet Connection for SMS Services (Optional).

CHAPTER 4

SYSTEM DESIGN AND DEVELOPMENT

4.1 METHODOLOGY

The methodology of the Smart Blood Donation and Emergency Request System involves a systematic approach to designing and developing an efficient platform for managing blood donation during emergencies. The process begins with system analysis, where the existing blood donation process is studied to identify issues such as delays in finding donors, lack of real-time communication, and difficulty in managing emergency requests. Based on this analysis, the system requirements are gathered, including functional requirements like donor registration, emergency blood request creation, donor matching based on blood group and location, sending notifications, and automatic request closure once the blood is arranged. Non-functional requirements such as fast response time, data security, and ease of use are also considered.

Once development is complete, the system undergoes testing to ensure proper functionality, including checking donor registration, request creation, matching accuracy, and notification delivery. Finally, the system is deployed as a web or desktop application. This methodology ensures that blood donation requests are handled quickly and efficiently, reducing delays and improving emergency response in critical situations.

4.2 SOURCE CODE

```
const express=require('express');  
  
const bodyParser=require('body-parser');  
  
const app=express();  
  
app.use(bodyParser.urlencoded({extended:true}));  
  
app.use(express.json());
```

```
// HOME PAGE
```

SMART BLOOD DONATION AND EMERGENCY REQUEST

```
app.get('/',(req,res)=>{
res.send(`
<!DOCTYPE html>

<html>

<head>

<title>Blood Donation</title>

<style>

body{margin:0;font-family:Arial;background:#f5f5f5}

header{background:darkred;color:#fff;padding:15px;text-align:center}

.hero{padding:100px;text-align:center;background:#b30000;color:#fff}

.btn{padding:12px 20px;background:#fff;color:darkred;text-decoration:none;border-radius:5px}

.section{padding:40px;text-align:center}

.blood{display:inline-block;width:70px;height:70px;line-
height:70px;background:darkred;color:#fff;border-radius:50%;margin:10px}

footer{background:#000;color:#fff;text-align:center;padding:15px}

</style>

</head>

<body>

<header>

<h1>Smart Blood Donation</h1>

<a href="/login" style="color:white">Login</a> |

<a href="/signup" style="color:white">Signup</a>

</header>
```

SMART BLOOD DONATION AND EMERGENCY REQUEST

```
<div class="hero">

<h1>Donate Blood Save Life</h1>

<p>Emergency Blood Donation Platform</p>

<a href="/donor" class="btn">Become Donor</a>

</div>

<div class="section">

<h2>Blood Groups</h2>

<div class="blood">A+</div>

<div class="blood">B+</div>

<div class="blood">O+</div>

<div class="blood">AB+</div>

</div>

<footer>

<p>Blood Donation System</p>

</footer>

</body>

</html>

`);

});
```

```
// LOGIN PAGE
```

```
app.get('/login',(req,res)=>{
```

SMART BLOOD DONATION AND EMERGENCY REQUEST

```
res.send(`
<!DOCTYPE html>

<html>

<head>

<title>Login</title>

<style>

body{font-family:Arial;background:#eee}

form{width:300px;margin:100px auto;background:white;padding:20px}

input,button{width:100%;padding:10px;margin:10px 0}

button{background:darkred;color:white;border:none}

</style>

</head>

<body>

<form>

<h2>Login</h2>

<input type="email" placeholder="Email">

<input type="password" placeholder="Password">

<button>Login</button>

</form>

</body>

</html>

`);
```

SMART BLOOD DONATION AND EMERGENCY REQUEST

```
});
```

```
// SIGNUP PAGE
```

```
app.get('/signup',(req,res)=>{
```

```
res.send(`
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Signup</title>
```

```
<style>
```

```
body{font-family:Arial;background:#eee}
```

```
form{width:300px;margin:50px auto;background:white;padding:20px}
```

```
input,select,button{width:100%;padding:10px;margin:10px 0}
```

```
button{background:darkred;color:white;border:none}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<form action="/register" method="POST">
```

```
<h2>Signup</h2>
```

```
<input type="text" name="name" placeholder="Name">
```

SMART BLOOD DONATION AND EMERGENCY REQUEST

```
<input type="email" name="email" placeholder="Email">

<select name="blood">

<option>A+</option>

<option>B+</option>

<option>O+</option>

<option>AB+</option>

</select>

<button>Register</button>

</form>

</body>

</html>

`);

});

// DONOR PAGE

app.get('/donor',(req,res)=>{

res.send(`

<h1 style="text-align:center;color:darkred;margin-top:100px">

Thank You For Becoming a Donor

</h1>

`);

});
```

SMART BLOOD DONATION AND EMERGENCY REQUEST

```
// REGISTER ROUTE

app.post('/register',(req,res)=>{

  console.log(req.body);

  res.send(`

    <h1 style="text-align:center;color:green">

    Registration Successful

    </h1>

  `);

});

// SERVER

app.listen(5000,()=>{

  console.log('Server Running on http://localhost:5000');

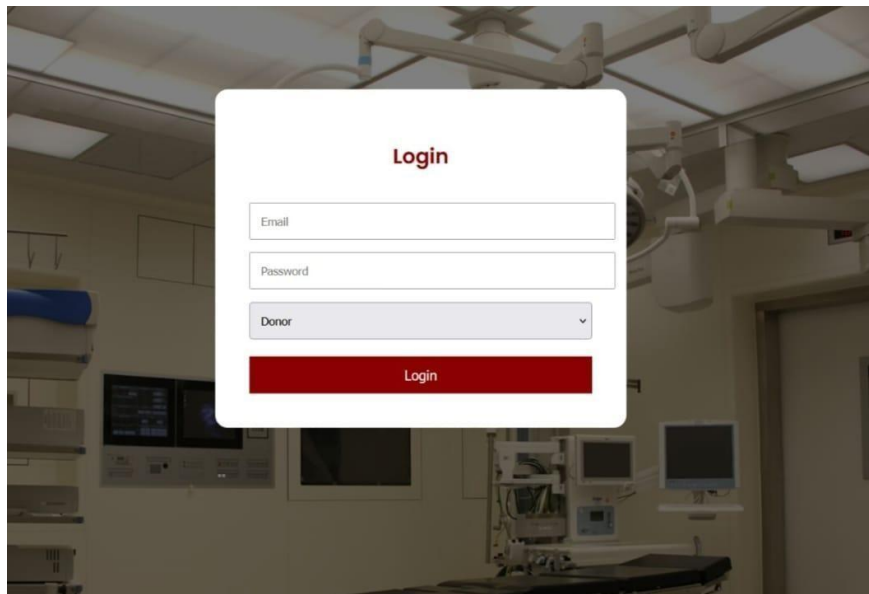
});
```

CHAPTER 5

RESULT AND INTERPRETATION

5.1 OUTCOMES

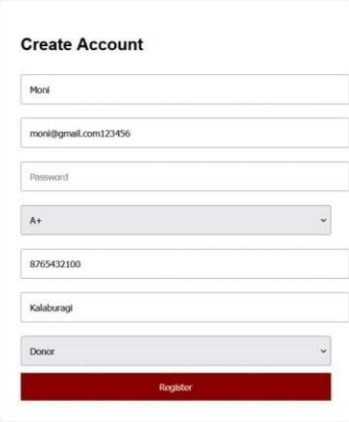
SNAPSHOT: 01



LOGIN PROCESS

SMART BLOOD DONATION AND EMERGENCY REQUEST

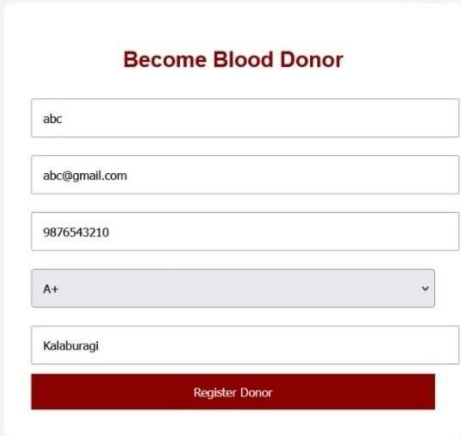
SNAPSHOT:02



The 'Create Account' form is a vertical stack of input fields. It starts with a title 'Create Account' in bold. Below it are fields for 'Name' (containing 'Moni'), 'Email' (containing 'moni@gmail.com123456'), 'Password', a blood type dropdown menu (set to 'A+'), a phone number field (containing '8765432100'), a location field (containing 'Kalaburagi'), and a donor type dropdown menu (set to 'Donor'). At the bottom is a red button labeled 'Register'.

CREATE ACCOUNT

SNAPSHOT:03



The 'Become Blood Donor' form is a vertical stack of input fields. It starts with a title 'Become Blood Donor' in bold. Below it are fields for 'Name' (containing 'abc'), 'Email' (containing 'abc@gmail.com'), a phone number field (containing '9876543210'), a blood type dropdown menu (set to 'A+'), and a location field (containing 'Kalaburagi'). At the bottom is a red button labeled 'Register Donor'.

DONOR REGISTRATION

SMART BLOOD DONATION AND EMERGENCY REQUEST

SNAPSHOT:04



The screenshot shows a web form titled "Emergency Blood Request". It contains five input fields: a text field with "xyz", a dropdown menu with "A+", a text field with "ESI Hospital, kalaburgi", a text field with "1234567890", and a text field with "ESI Medical college and hospital ,Kalaburagi". Below these fields is a red button labeled "Send Request".

SEND REQUEST

CONCLUSION

The Smart Blood Donation and Emergency Request System provides an efficient and reliable solution for managing blood donation during critical situations. By automating donor registration, emergency request handling, and real-time donor matching based on blood group and location, the system significantly reduces the time required to find suitable donors. The notification feature ensures quick communication between patients and donors, improving the chances of saving lives in emergencies. Additionally, the automatic request closure feature helps maintain updated and accurate records. Overall, this system enhances coordination between donors and recipients, making the blood donation process faster, more organized, and more effective in addressing urgent medical needs.

REFERENCE

[1] Optimization of blood donation activity supporting a smart city.

Published in: Applied Medical Informatics , 2018, Directory of open access journals.

By: Anca-Elena ; Adrina Albu.

[2] Smart approaches for encouraging the blood donation.

Published in: Asian journal of transfusion science , 2024 , directory of open access journals.

By: Rakesh Sharma; Debadri Banerjee; Anupama Singh; Vikas Anand Saharan.

[3]. Design of a smart blood donation assistant based on large language model.

Published in: Zhongguo shuxue zazhi, 2026, Directory of open access journals.

By: LUN Lan; WAN Kangile; ZHENG Yue; ZHAO Xiaoya; HAN Zhedong.